

Department of Software Engineering

Demonstration of the Rent-a-Car Management System

Submitted by:

Muhammad Aliyan

25021598-245

Ghulam Meeran

25021598-191

Abdul Mateen

25021598-259

Jahanzaib Khalid

25021598-180

Section:

AB

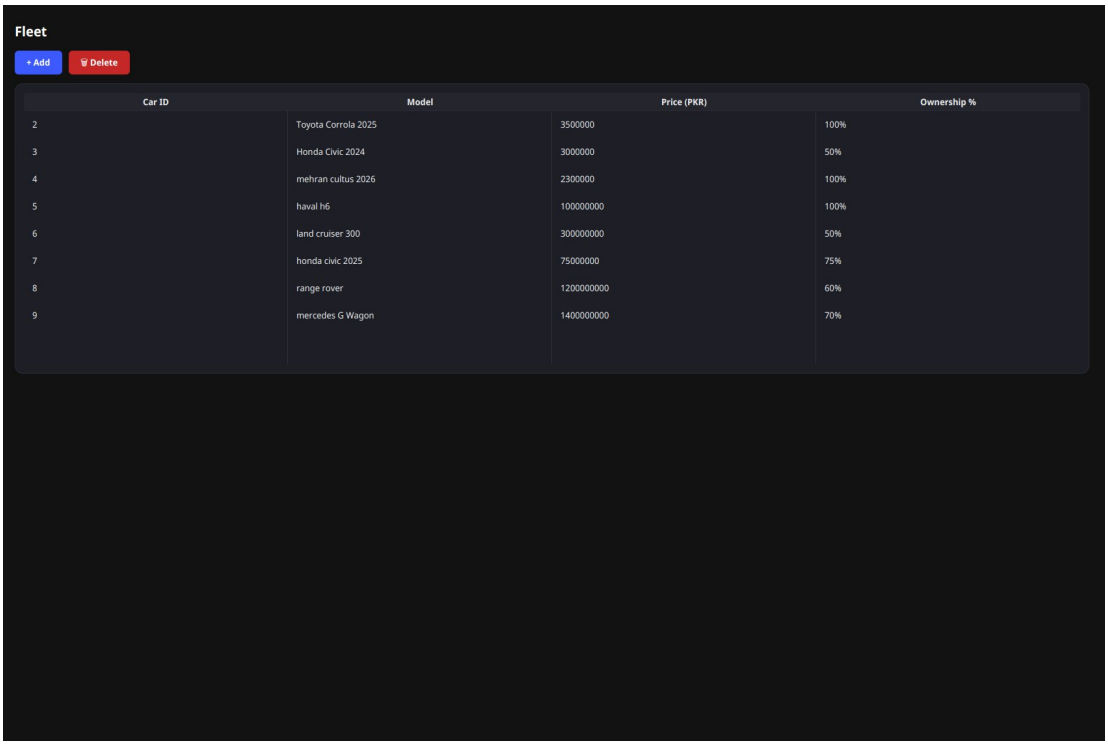
Submitted to:

Sir Muhammad Ejaz

Table of Contents

Fleet **3**
Vehicle.java 3
VehicleDAO.java 4
Customers **6**
Customers.java 6
CustomerDAO.java 7
Drivers **9**
Driver.java 9
DriverDAO.java 10
Sales **12**
Sale.java 12
SaleDAO.java 14
Payments **18**
Payment.java 18
PaymentDAO.java 19
Expenses **21**
Expense.java 21
ExpenseDAO.java 22

Fleet



Vehicle.java

```
package models;

public class Vehicle {
    private int carId;
    private String model;
    private String purchasePrice;
    private double ownershipPercentage;

    public Vehicle(int carId, String model, String purchasePrice, double ownershipPercentage) {
        this.carId = carId;
        this.model = model;
        this.purchasePrice = purchasePrice;
        this.ownershipPercentage = ownershipPercentage;
    }

    public int getCarId() {
        return carId;
    }

    public String getModel() {
        return model;
```

```
    }

    public String getPurchasePrice() {
        return purchasePrice;
    }

    public double getOwnershipPercentage() {
        return ownershipPercentage;
    }
}
```

VehicleDAO.java

```
package database;

import models.Vehicle;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class VehicleDAO {

    public List<Vehicle> listVehicles() throws SQLException {
        String sql = "SELECT car_ID, model, purchase_Price, ownership_Percentage FROM Vehicles";

        try (
                Connection conn = DBConfig.getConnection();
                PreparedStatement ps = conn.prepareStatement(sql);
                ResultSet rs = ps.executeQuery()) {
            List<Vehicle> vehicles = new ArrayList<>();

            while (rs.next()) {
                vehicles.add(new Vehicle(
                        rs.getInt("car_ID"),
                        rs.getString("model"),
                        rs.getString("purchase_Price"),
                        rs.getDouble("ownership_Percentage")));
            }

            return vehicles;
        }
    }
```

```
    public void insert(String model, String purchasePrice, double ownershipPercentage) throws
SQLException {
        String sql = "INSERT INTO Vehicles (model, purchase_Price, ownership_Percentage) VALUES
(?, ?, ?)";
        try (Connection conn = DBConfig.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, model);
            ps.setString(2, purchasePrice);
            ps.setDouble(3, ownershipPercentage);
            ps.executeUpdate();
        }
    }

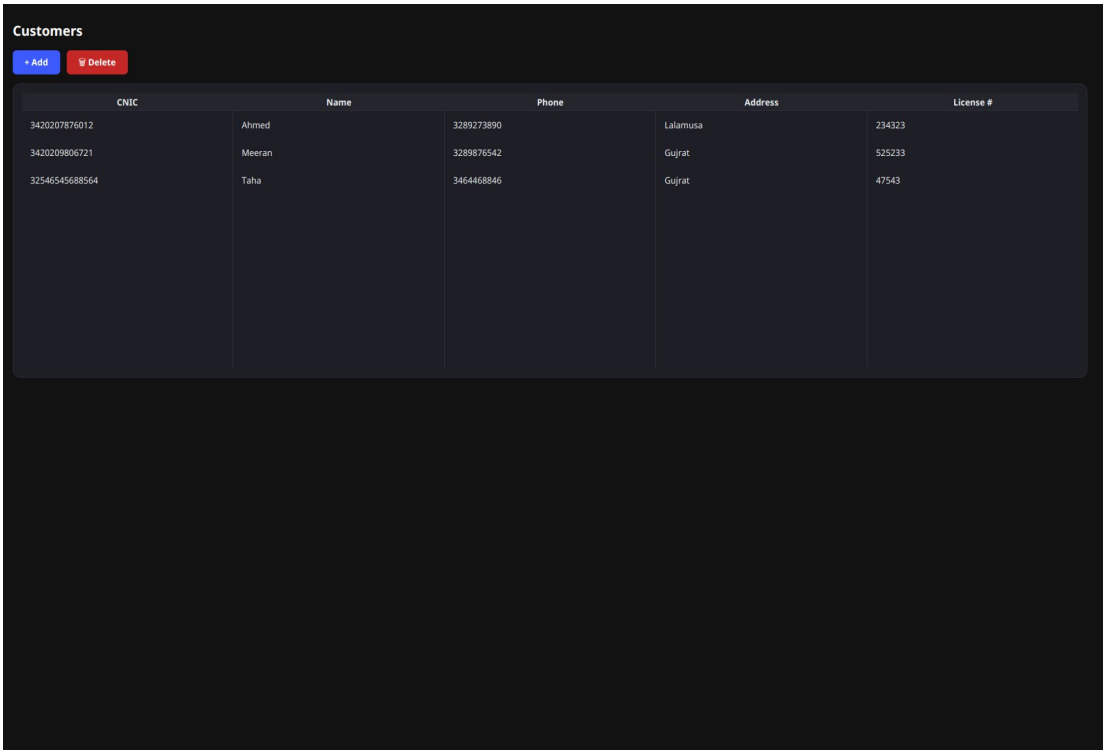
    public void delete(int carId) throws SQLException {
        try (Connection conn = DBConfig.getConnection()) {
            conn.setAutoCommit(false);

            try (PreparedStatement psCheck = conn.prepareStatement(
                    "SELECT COUNT(*) FROM Sales WHERE car_id = ?")) {
                psCheck.setInt(1, carId);
                try (ResultSet rs = psCheck.executeQuery()) {
                    if (rs.next() && rs.getInt(1) > 0) {
                        conn.rollback();
                        throw new SQLException("Cannot delete vehicle: there are sales referencing this
vehicle. Remove or reassign those sales first.");
                    }
                }
            }

            try (PreparedStatement psClear = conn.prepareStatement(
                    "UPDATE Expenses SET vehicle_id = NULL WHERE vehicle_id = ?")) {
                psClear.setInt(1, carId);
                psClear.executeUpdate();
            }

            try (PreparedStatement ps = conn.prepareStatement(
                    "DELETE FROM Vehicles WHERE car_ID = ?")) {
                ps.setInt(1, carId);
                ps.executeUpdate();
            }
            conn.commit();
        }
    }
}
```

Customers



Customers.java

```
package models;

public class Customer {
    private long cnic;
    private String name;
    private String address;
    private long phone;
    private String license;

    public Customer(long cnic, String name, String address, long phone, String license) {
        this.cnic = cnic;
        this.name = name;
        this.address = address;
        this.phone = phone;
        this.license = license;
    }

    public long getCnic() {
        return cnic;
    }
```

Customers

```
    public String getName() {
        return name;
    }

    public long getPhone() {
        return phone;
    }

    public String getAddress() {
        return address;
    }

    public String getLicense() {
        return license;
    }
}
```

CustomerDAO.java

```
package database;

import models.Customer;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class CustomerDAO {

    public List<Customer> listCustomers() throws SQLException {
        String sql = "SELECT customer_cnic, custome_name, address, phone_number,
license_number FROM Customers";

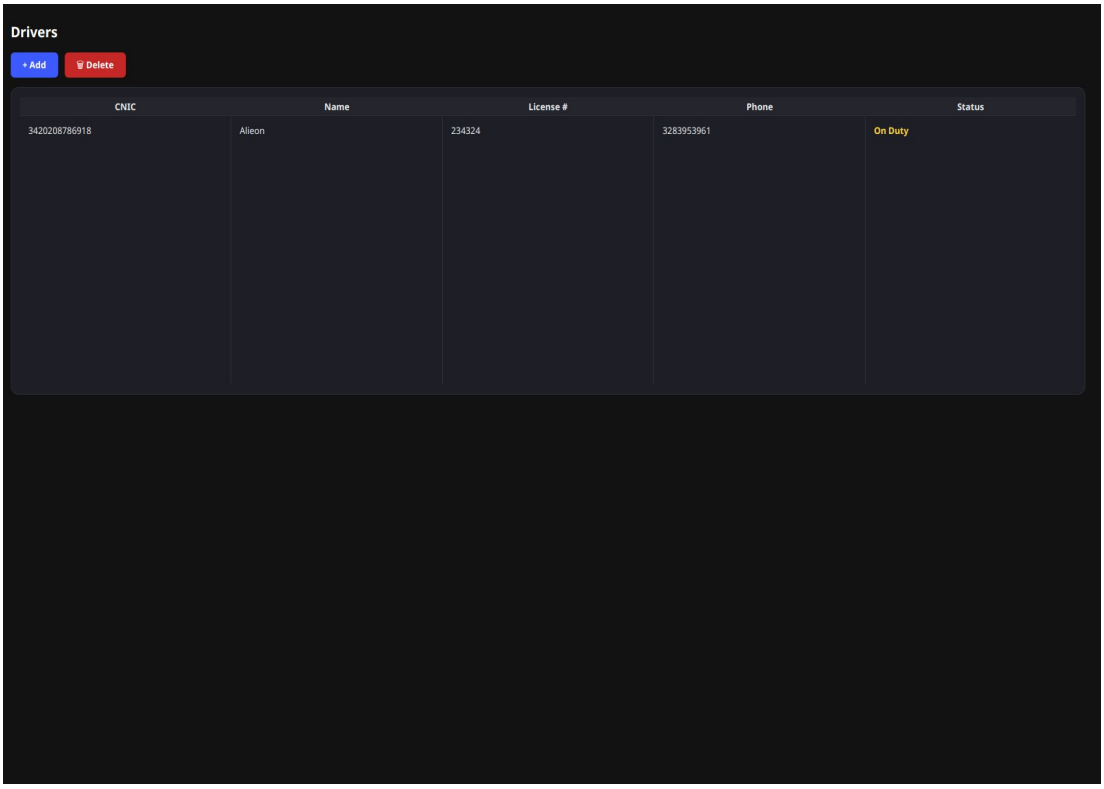
        try (
                Connection conn = DBConfig.getConnection();
                PreparedStatement ps = conn.prepareStatement(sql);
                ResultSet rs = ps.executeQuery()) {
            List<Customer> customers = new ArrayList<>();
            while (rs.next()) {
                customers.add(new Customer(
```

```
                rs.getLong("customer_cnic"),
                rs.getString("custome_name"),
                rs.getString("address"),
                rs.getLong("phone_number"),
                rs.getString("license_number")));
        }
        return customers;
    }
  }

  public void insert(long cnic, String name, String address, long phone, String license) throws
SQLException {
    String sql = "INSERT INTO Customers (customer_cnic, custome_name, address,
phone_number, license_number) VALUES (?, ?, ?, ?, ?)";
    try (Connection conn = DBConfig.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, cnic);
        ps.setString(2, name);
        ps.setString(3, address);
        ps.setLong(4, phone);
        ps.setString(5, license);
        ps.executeUpdate();
    }
  }

  public void delete(long cnic) throws SQLException {
    String sql = "DELETE FROM Customers WHERE customer_cnic = ?";
    try (Connection conn = DBConfig.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setLong(1, cnic);
        ps.executeUpdate();
    }
  }
}
```

Drivers



Driver.java

```
package models;

public class Driver {
    private long cnic;
    private String name;
    private long license;
    private long phone;
    private String status;

    public Driver(long cnic, String name, long license, long phone, String status) {
        this.cnic = cnic;
        this.name = name;
        this.license = license;
        this.phone = phone;
        this.status = status;
    }

    public long getCnic() {
        return cnic;
```

```
    }

    public String getName() {
        return name;
    }

    public long getLicense() {
        return license;
    }

    public long getPhone() {
        return phone;
    }

    public String getStatus() {
        return status;
    }
}
```

DriverDAO.java

```
package database;

import models.Driver;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class DriverDAO {

    public List<Driver> listDrivers() throws SQLException {
        String sql = "SELECT d.driver_cnic, d.driver_name, d.license_number, d.phone_number, " +
                " CASE WHEN active.driver_cnic IS NOT NULL THEN 'On Duty' " +
                " ELSE d.status END AS eff_status " +
                "FROM Drivers d " +
                "LEFT JOIN ( " + " SELECT DISTINCT driver_cnic FROM Sales " +
                " WHERE end_date >= CURDATE() AND driver_cnic > 0 " +
                ") active ON active.driver_cnic = d.driver_cnic";
```

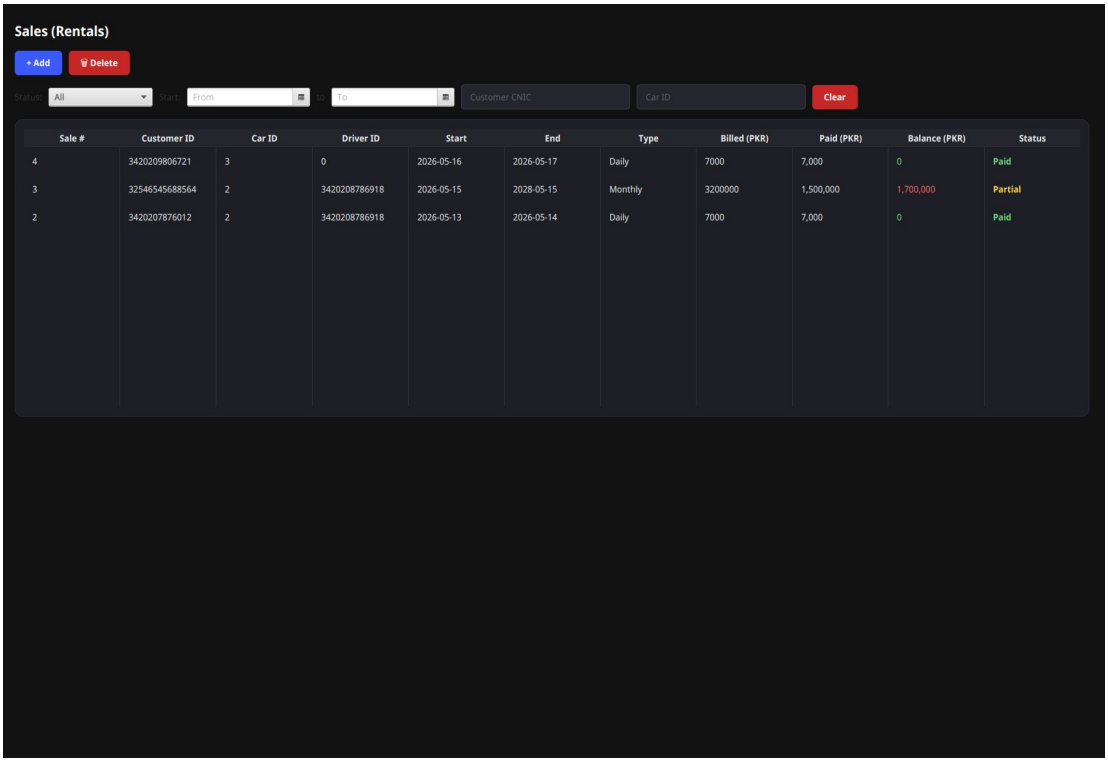
Drivers

```
        try (
                Connection conn = DBConfig.getConnection();
                PreparedStatement ps = conn.prepareStatement(sql);
                ResultSet rs = ps.executeQuery()) {
            List<Driver> drivers = new ArrayList<>();
            while (rs.next()) {
                drivers.add(new Driver(
                        rs.getLong("driver_cnic"),
                        rs.getString("driver_name"),
                        rs.getLong("license_number"),
                        rs.getLong("phone_number"),
                        rs.getString("eff_status")));
            }
            return drivers;
        }
    }

    public void insert(long cnic, String name, long license, long phone, String status) throws
SQLException {
        String sql = "INSERT INTO Drivers (driver_cnic, driver_name, license_number, phone_number,
status) VALUES (?, ?, ?, ?, ?)";
        try (Connection conn = DBConfig.getConnection();
                PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, cnic);
            ps.setString(2, name);
            ps.setLong(3, license);
            ps.setLong(4, phone);
            ps.setString(5, status);
            ps.executeUpdate();
        }
    }

    public void delete(long cnic) throws SQLException {
        String sql = "DELETE FROM Drivers WHERE driver_cnic = ?";
        try (Connection conn = DBConfig.getConnection();
                PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, cnic);
            ps.executeUpdate();
        }
    }
}
```

Sales



Sale.java

```
package models;

import java.time.LocalDate;

public class Sale {
    private int saleId;
    private long customerCnic;
    private int carId;
    private long driverCnic;
    private LocalDate startDate;
    private LocalDate endDate;
    private String rentalType;
    private int totalAmount;

    public Sale(long customerCnic, int carId, long driverCnic,
            LocalDate start, LocalDate end, String type, int total) {
        this.customerCnic = customerCnic;
        this.carId = carId;
        this.driverCnic = driverCnic;
        this.startDate = start;
```

```
    this.endDate = end;
    this.rentalType = type;
    this.totalAmount = total;
}

public Sale(int saleId, long customerCnic, int carId, long driverCnic,
        LocalDate start, LocalDate end, String type, int total) {
    this(customerCnic, carId, driverCnic, start, end, type, total);
    this.saleId = saleId;
}

private double amountPaid = -1;
private double balance = -1;
private String paymentStatus = "";

public void setAmountPaid(double v) {
    this.amountPaid = v;
}

public void setBalance(double v) {
    this.balance = v;
}

public void setPaymentStatus(String v) {
    this.paymentStatus = v;
}

public int getSaleId() {
    return saleId;
}

public long getCustomerCnic() {
    return customerCnic;
}

public int getCarId() {
    return carId;
}

public long getDriverCnic() {
    return driverCnic;
}

public LocalDate getStartDate() {
    return startDate;
```

```
    }

    public LocalDate getEndDate() {
        return endDate;
    }

    public String getRentalType() {
        return rentalType;
    }

    public int getTotalAmount() {
        return totalAmount;
    }

    public double getAmountPaid() {
        return amountPaid;
    }

    public double getBalance() {
        return balance;
    }

    public String getPaymentStatus() {
        return paymentStatus;
    }

    public boolean isOverdue() {
        return endDate != null && endDate.isBefore(LocalDate.now())
                && !"Paid".equals(paymentStatus);
    }
}
```

SaleDAO.java

```
package database;

import models.Sale;

import java.sql.Connection;
import java.sql.Date;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
```

Sales

```
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class SaleDAO {

   public List<Sale> listSales() throws SQLException {
      String sql = "SELECT sale_id, customer_cnic, car_id, driver_cnic,  start_date, end_date,
rental_type, Total FROM Sales";

      try (
            Connection conn = DBConfig.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
         List<Sale> sales = new ArrayList<>();
         while (rs.next()) {
            Date start = rs.getDate("start_date");
            Date end = rs.getDate("end_date");

            sales.add(new Sale(
                  rs.getInt("sale_id"),
                  rs.getLong("customer_cnic"),
                  rs.getInt("car_id"),
                  rs.getLong("driver_cnic"),
                  start != null ? start.toLocalDate() : null,
                  end != null ? end.toLocalDate() : null,
                  rs.getString("rental_type"),
                  rs.getInt("Total")));
         }
         return sales;
      }
   }

   public void insert(long customerCnic, int carId, long driverCnic,
         LocalDate start, LocalDate end, String rentalType, int total) throws SQLException {
      String sql = "INSERT INTO Sales (customer_cnic, car_id, driver_cnic, start_date, end_date,
rental_type, Total) VALUES (?, ?, ?, ?, ?, ?, ?)";
      try (Connection conn = DBConfig.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
         ps.setLong(1, customerCnic);
         ps.setInt(2, carId);
         if (driverCnic > 0)
            ps.setLong(3, driverCnic);
         else
            ps.setNull(3, java.sql.Types.BIGINT);
```

```
        ps.setDate(4, start != null ? Date.valueOf(start) : null);
        ps.setDate(5, end != null ? Date.valueOf(end) : null);
        ps.setString(6, rentalType);
        ps.setInt(7, total);
        ps.executeUpdate();
    }
}

public void delete(int saleId) throws SQLException {
    String sql = "DELETE FROM Sales WHERE sale_id = ?";
    try (Connection conn = DBConfig.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, saleId);
        ps.executeUpdate();
    }
}

public List<Sale> listSalesWithBalance() throws SQLException {
    String sql = "SELECT s.sale_id, s.customer_cnic, s.car_id, s.driver_cnic, " +
            "s.start_date, s.end_date, s.rental_type, s.Total, " +
            "COALESCE(SUM(p.money_paid), 0) AS amount_paid " +
            "FROM Sales s " +
            "LEFT JOIN Payment p ON p.sale_id = s.sale_id " +
            "GROUP BY s.sale_id, s.customer_cnic, s.car_id, s.driver_cnic, " +
            "s.start_date, s.end_date, s.rental_type, s.Total " +
            "ORDER BY s.sale_id DESC";

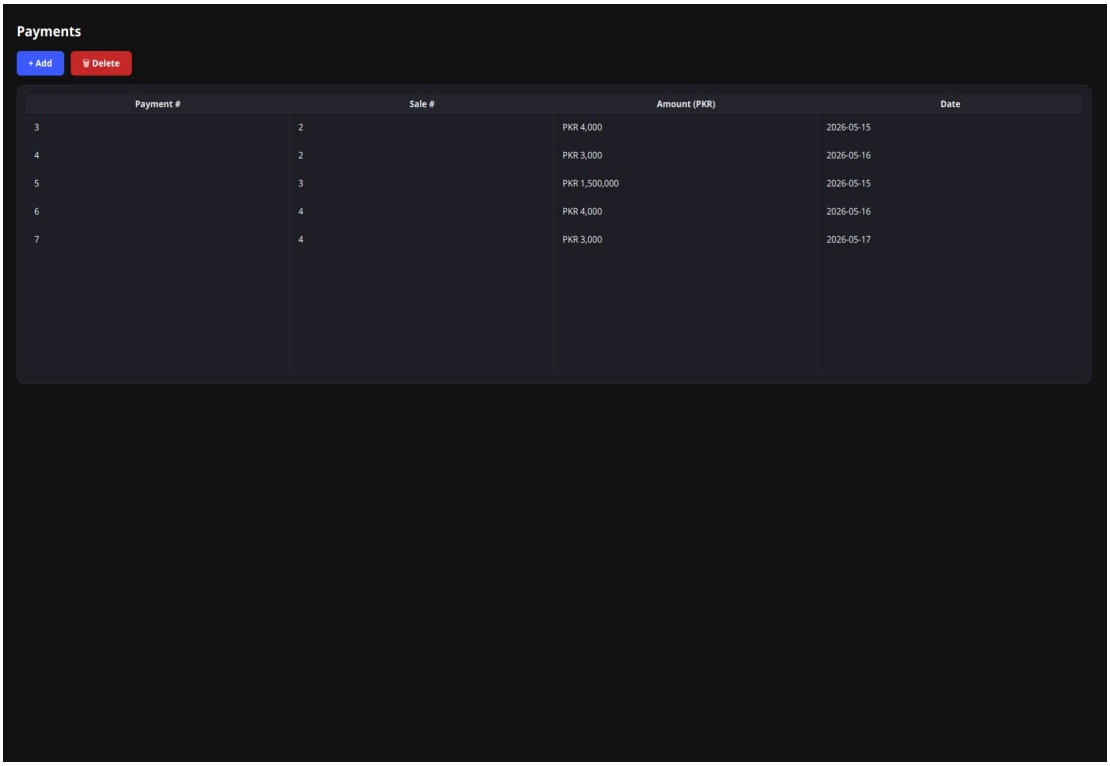
    try (Connection conn = DBConfig.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {

        List<Sale> list = new ArrayList<>();
        while (rs.next()) {
            Date start = rs.getDate("start_date");
            Date end = rs.getDate("end_date");
            int total = rs.getInt("Total");
            double paid = rs.getDouble("amount_paid");

            Sale sale = new Sale(
                    rs.getInt("sale_id"),
                    rs.getLong("customer_cnic"),
                    rs.getInt("car_id"),
                    rs.getLong("driver_cnic"),
                    start != null ? start.toLocalDate() : null,
                    end != null ? end.toLocalDate() : null,
```

```
                    rs.getString("rental_type"),
                    total);
            sale.setAmountPaid(paid);
            sale.setBalance(total - paid);
            if (paid >= total)
                sale.setPaymentStatus("Paid");
            else if (paid > 0)
                sale.setPaymentStatus("Partial");
            else
                sale.setPaymentStatus("Unpaid");
            list.add(sale);
        }
        return list;
    }
  }
}
```

Payments



Payment.java

```
package models;

import java.time.LocalDate;

public class Payment {
    private int paymentId;
    private int saleId;
    private double moneyPaid;
    private LocalDate paymentDate;

    public Payment(int paymentId, int saleId, double moneyPaid, LocalDate paymentDate) {
        this.paymentId = paymentId;
        this.saleId = saleId;
        this.moneyPaid = moneyPaid;
        this.paymentDate = paymentDate;
    }

    public int getPaymentId() {
        return paymentId;
```

```
    }

    public int getSaleId() {
        return saleId;
    }

    public double getMoneyPaid() {
        return moneyPaid;
    }

    public LocalDate getPaymentDate() {
        return paymentDate;
    }
}
```

PaymentDAO.java

```
package database;

import models.Payment;

import java.sql.Connection;
import java.sql.Date;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class PaymentDAO {
    public List<Payment> listPayments() throws SQLException {
        String sql = "SELECT payment_id, sale_id, money_paid, payment_date FROM Payment";

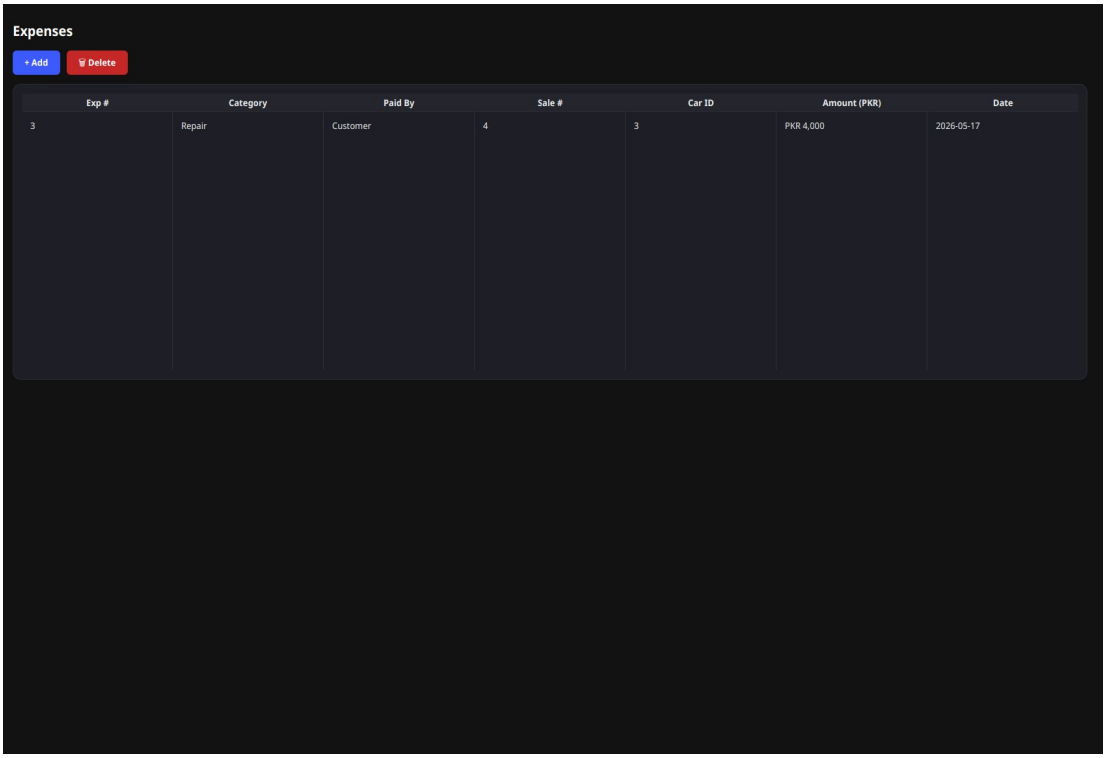
        try (
                Connection conn = DBConfig.getConnection();
                PreparedStatement ps = conn.prepareStatement(sql);
                ResultSet rs = ps.executeQuery()) {
            List<Payment> payments = new ArrayList<>();
            while (rs.next()) {
                Date d = rs.getDate("payment_date");
                payments.add(new Payment(
                        rs.getInt("payment_id"),
```

```
                rs.getInt("sale_id"),
                rs.getDouble("money_paid"),
                d != null ? d.toLocalDate() : null));
        }
        return payments;
    }
}

public void insert(int saleId, double moneyPaid, LocalDate date) throws SQLException {
    String sql = "INSERT INTO Payment (sale_id, money_paid, payment_date) VALUES (?, ?, ?)";
    try (Connection conn = DBConfig.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, saleId);
        ps.setDouble(2, moneyPaid);
        ps.setDate(3, date != null ? Date.valueOf(date) : null);
        ps.executeUpdate();
    }
}

public void delete(int paymentId) throws SQLException {
    String sql = "DELETE FROM Payment WHERE payment_id = ?";
    try (Connection conn = DBConfig.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, paymentId);
        ps.executeUpdate();
    }
}
}
```

Expenses



Expense.java

```
package models;

import java.time.LocalDate;

public class Expense {
    private int expenseId;
    private String type;
    private String payer;
    private int saleId;
    private int vehicleId;
    private double amount;
    private LocalDate date;

    public Expense(int expenseId, String type, String payer,
            int saleId, int vehicleId, double amount, LocalDate date) {
        this.expenseId = expenseId;
        this.type = type;
        this.payer = payer;
        this.saleId = saleId;
        this.vehicleId = vehicleId;
        this.amount = amount;
        this.date = date;
```

```
    }

    public int getExpenseId() {
        return expenseId;
    }

    public String getType() {
        return type;
    }

    public String getPayer() {
        return payer;
    }

    public int getSaleId() {
        return saleId;
    }

    public int getVehicleId() {
        return vehicleId;
    }

    public double getAmount() {
        return amount;
    }

    public LocalDate getDate() {
        return date;
    }
}
```

ExpenseDAO.java

```
package database;

import models.Expense;

import java.sql.Connection;
import java.sql.Date;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
```

Expenses

```
public class ExpenseDAO {
   public List<Expense> listExpenses() throws SQLException {
      String sql = "SELECT expense_id, type, payer, sale_id, vehicle_id, amount, date FROM
Expenses";

      try (
            Connection conn = DBConfig.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
         List<Expense> expenses = new ArrayList<>();
         while (rs.next()) {
            Date d = rs.getDate("date");
            expenses.add(new Expense(
                  rs.getInt("expense_id"),
                  rs.getString("type"),
                  rs.getString("payer"),
                  rs.getInt("sale_id"),
                  rs.getInt("vehicle_id"),
                  rs.getDouble("amount"),
                  d != null ? d.toLocalDate() : null));
         }
         return expenses;
      }
   }

   public void insert(String type, String payer, int saleId, int vehicleId, double amount, LocalDate
date)
         throws SQLException {
      String sql = "INSERT INTO Expenses (type, payer, sale_id, vehicle_id, amount, date) VALUES
(?, ?, ?, ?, ?, ?)";
      try (Connection conn = DBConfig.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
         ps.setString(1, type);
         ps.setString(2, payer);
         if (saleId > 0)
            ps.setInt(3, saleId);
         else
            ps.setNull(3, java.sql.Types.INTEGER);
         if (vehicleId > 0)
            ps.setInt(4, vehicleId);
         else
            ps.setNull(4, java.sql.Types.INTEGER);
         ps.setDouble(5, amount);
         ps.setDate(6, date != null ? Date.valueOf(date) : null);
```

```
            ps.executeUpdate();
        }
    }

    public void delete(int expenseId) throws SQLException {
        String sql = "DELETE FROM Expenses WHERE expense_id = ?";
        try (Connection conn = DBConfig.getConnection();
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, expenseId);
            ps.executeUpdate();
        }
    }
}
```